

Dependencies:

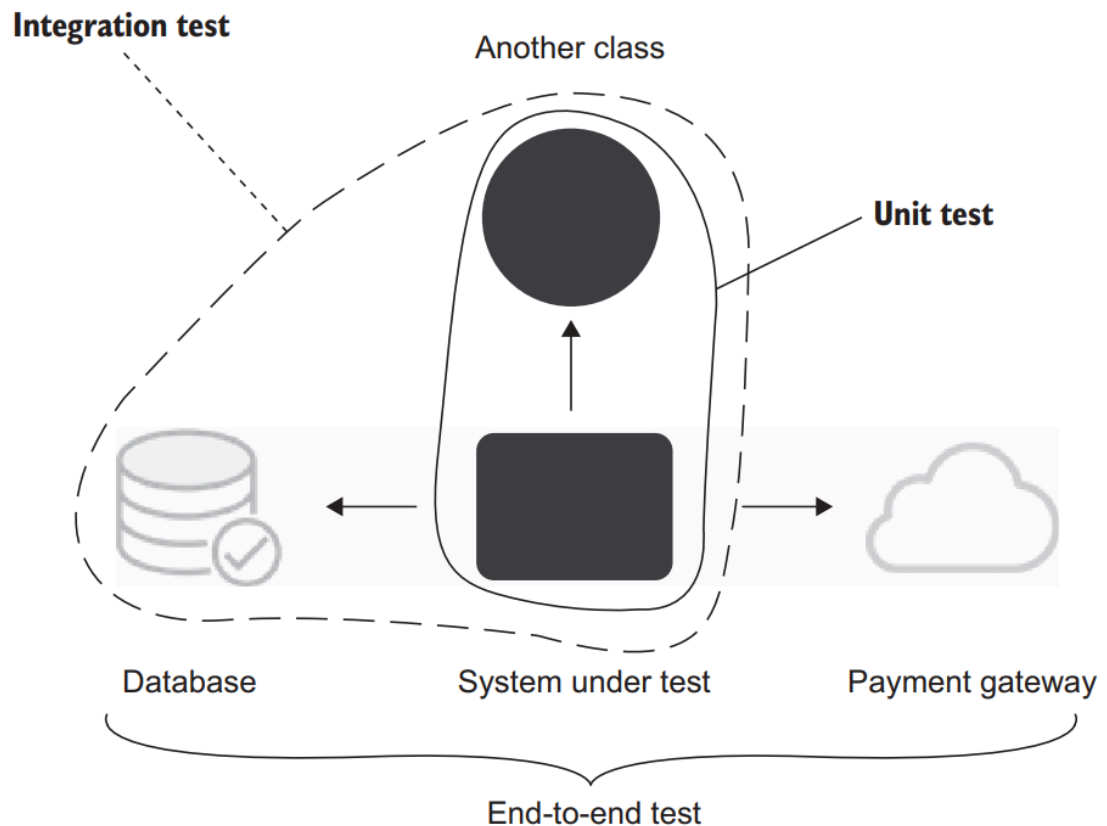
- *Shared dependency*—A dependency shared by tests (not production code)
- *Out-of-process dependency*—A dependency hosted by a process other than the program's execution process (for example, a database, a message bus, or an SMTP service)
- *Private dependency*—Any dependency that is not shared

Unit test :

- Verifies a single unit of behavior or an interface contract
- Does it quickly
- And does it in isolation from other tests

Integration test :

does the opposite but mocks third-party-services



Classicists

- mocks only shared among tests dependencies and out-of-process third-party services

Mockists:

- mocks everything

Table 5.2 The differences between the London and classical schools of unit testing

	Isolation of	A unit is	Uses test doubles for
London school	Units	A class	All but immutable dependencies
Classical school	Unit tests	A class or a set of classes	Shared dependencies

AAA

- Can create base class for entities to compare returned object to expected one
- avoid repeating a's (in integration test there could be cases where you connecting multiple tests in one to do more quickly, especially if arranges flows to one from another)
- no if statements in tests
- teardown fase can be in private method (maybe constructed into integration test base class) integration test base class with db and dispose method in 3 chapter
- can remove comments with aaa if there are no spaces inside those sections'
- Can start from assertions if doing tdd

Naming:

- Describe a unit of behaviour, a fact that is true (without should be , without unnecessary words) from the side of the client **The only exception to this guideline is when you work on utility code. Such code doesn't contain business logic—its behavior doesn't go much beyond simple auxiliary functionality and thus doesn't mean anything to business people**
public void User_passing_an_existing_name_cant_register
without returns badrequest or smth that describes implementation
- Even if test class named after some class which method being executed, this class is just an entrypoint to some behaviour
- SUT should be named as *sut*
- dont include sut name into test name

Reusing test fixtures between tests

- Create private methods for repeating or volatile test setups(==if setup is needed for multiple test classes then you can create object mother ==)
- High coupling between tests is an anti-pattern
- Constructors for setup diminishes test readability

Parameterized tests

- keep positive and negative cases in one test only if it self-evident for what case what inputs stands, if not , extract positive behaviour into sepearte test
- if behaviour is too complicated - dont use parameters

- Use [MemberData] to create input parameters with datetimenow

```
[Theory]
[MemberData(nameof(Data))]
public void Can_detect_an_invalid_delivery_date(
    DateTime deliveryDate,
    bool expected)
{
    /* ... */
}

public static List<object[]> Data()
{
    return new List<object[]>
    {
        new object[] { DateTime.Now.AddDays(-1), false },
        new object[] { DateTime.Now, false },
        new object[] { DateTime.Now.AddDays(1), false },
        new object[] { DateTime.Now.AddDays(2), true }
    };
}
```

or it wont work

Use Fluent Assertions

****The four pillars of a good unit test**

What are good Tests:

- Find bugs
- High resistance to refactoring
- working fast
- Easy to maintain

****The more code being executed in test, more chance that test will find bug**

****Refactoring means changing implementation not the functionality**

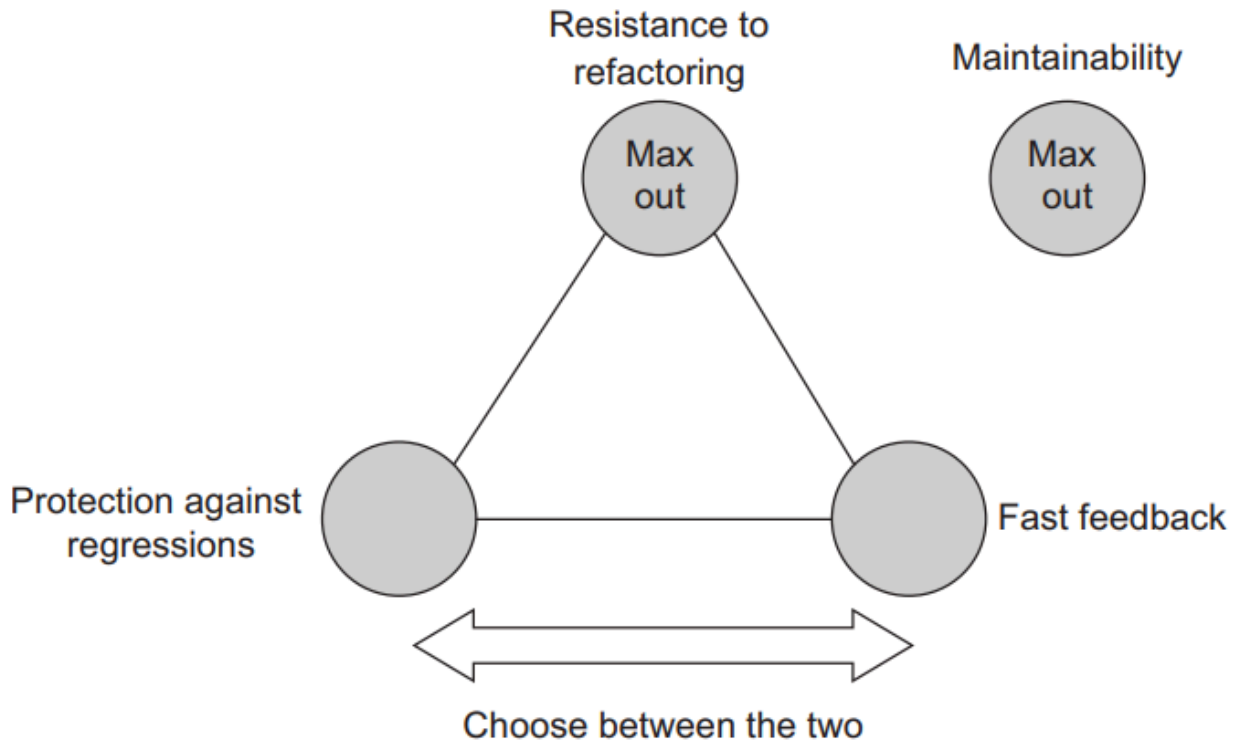
- You should test interface behaviour, not implementation
- tests must perceive sut like black-box
- **Resistance to refactoring is a low count of False positive* results*
- It is not such important at the beginning of the project as when the codebase grows

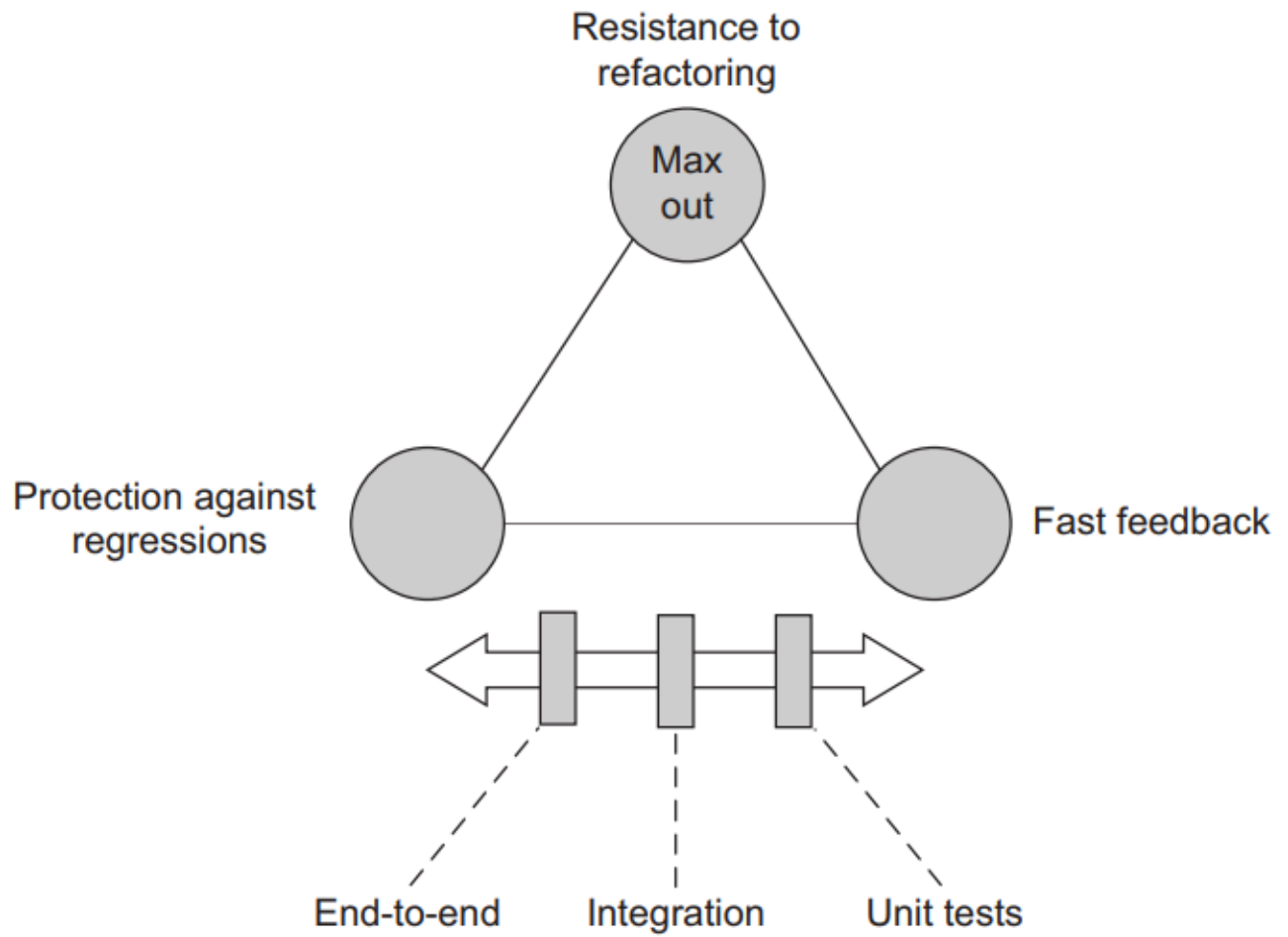
****Protection against regressions and resistance to refactoring contribute to test accuracy**

****Maintainability means:**

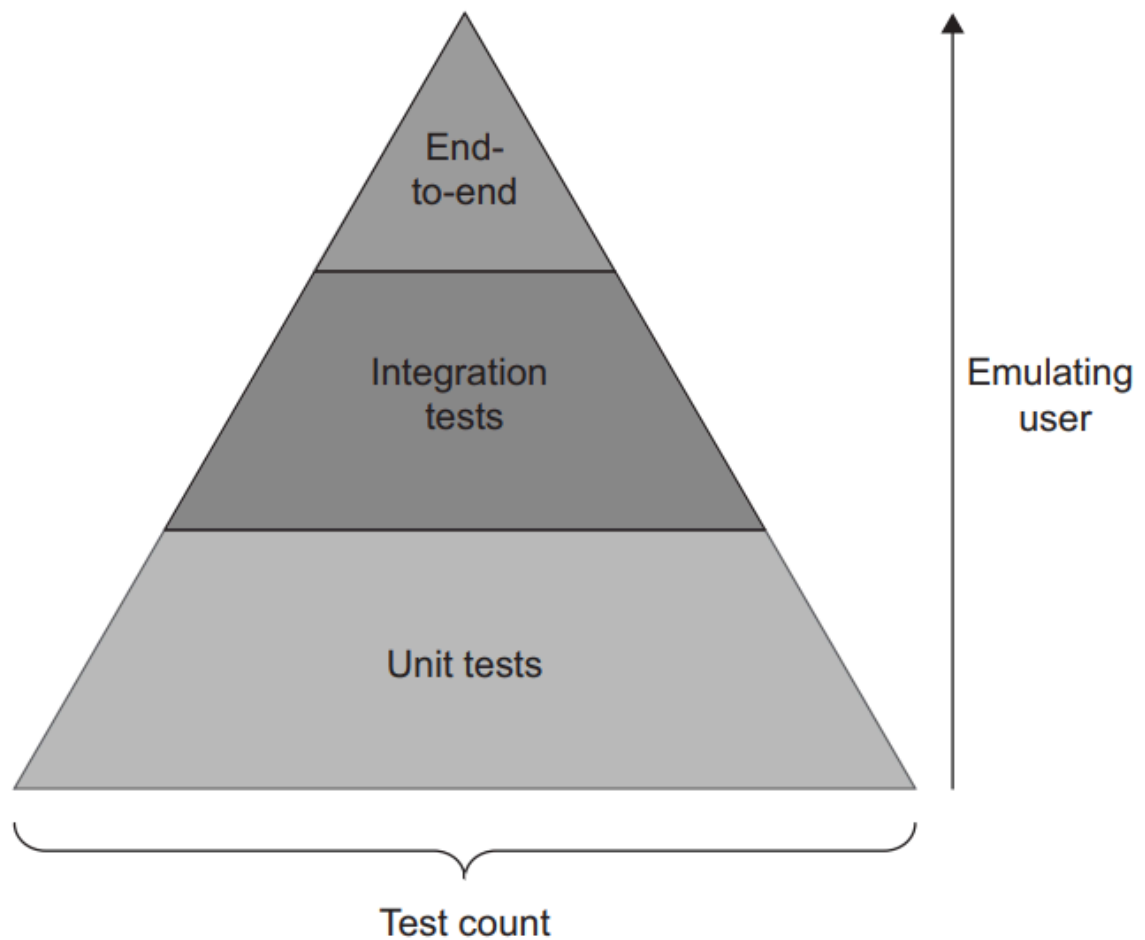
- How hard it is to understand the test, which is a function of the test's size
- How hard it is to run the test, which is a function of how many out-of-process dependencies the test works with directly
- ****You can always max maintainability and resistance to refactoring**
- **But you should choose between *Fast feedback* and *Ability to find bugs***

CHAPTER 4 *The four pillars of a good unit test*





Exploring well-known test automation concepts**:



- End-to-End tests should be passed in areas where you definitely don't want to see any bug and after all other tests

Exception :

- If your application is simple CRUD and no complex logic - your pyramid is a rectangle without end-to-end and with the same count of integration and unit tests (because unit tests are more valuable testing complex logic but integration tests are still important)
- If application is an API without user interface and with one external dependency (db) you can do more end-to-end tests and they will be fast
- **End-to-end differs from integration with entry point - they need application to be hosted somewhere imitating environment in which users actually can access your application via network
- **Integration tests mostly run on local or isolated environment

NEPONELNOOK :

Note that even though black-box testing is preferable when *writing tests*, you can still use the white-box method when *analyzing* the tests. *Use code coverage tools to see which code branches are not exercised, but then turn around and test them as if you know nothing about the code's internal structure.* Such a combination of the white-box and black-box methods works best.

Maybe about exercising all possible branches of code

****Mocks and test fragility**

This chapter covers

- Differentiating mocks from stubs
- Defining observable behavior and implementation details
- Understanding the relationship between mocks and test fragility
- Using mocks without compromising resistance to refactoring

Mocks and Stubs

- Mock is a test double used in outcome interactions with sut, which are the part of ending result of interface method
- Stub is a test double used in income interactions with sut which are some settings of necessary data and part of implementation

Never Assert stubs

SOME shistics

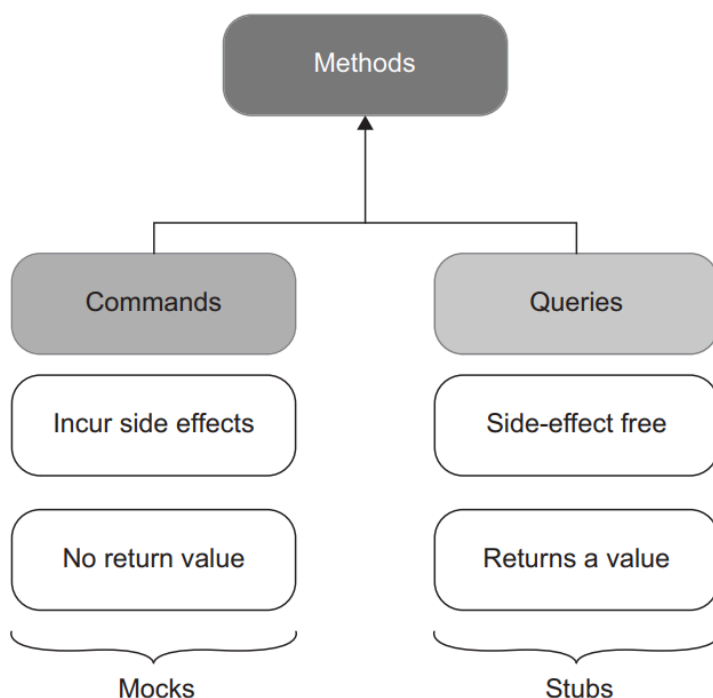
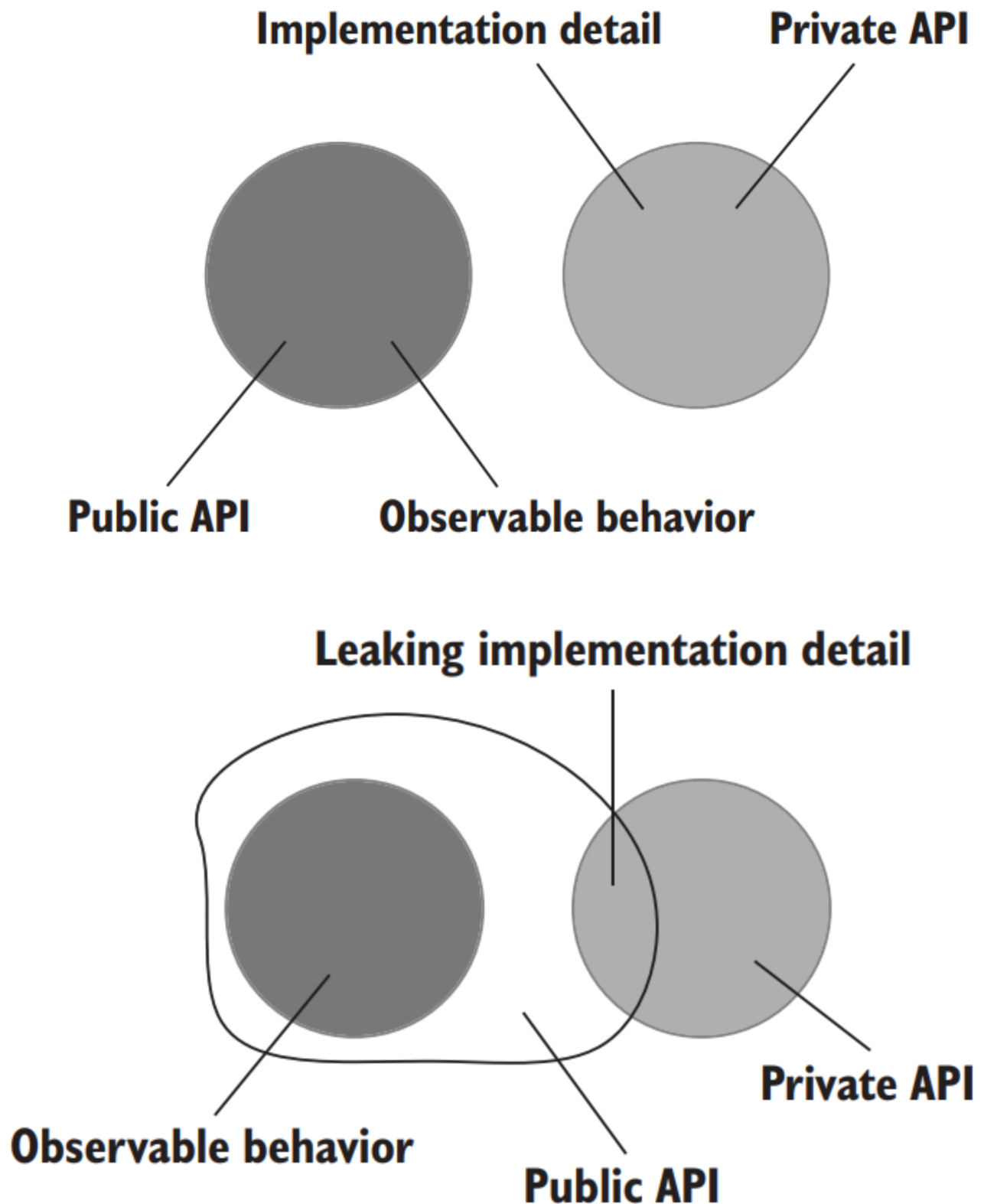


Figure 5.3 In the command query separation (CQS) principle, commands correspond to mocks, while queries are consistent with stubs.

Observable behavior vs. implementation details

Ideally, the system's public API surface should coincide with its observable behavior, and all its implementation details should be hidden from the eyes of the clients. Such a system has a well-designed API (



Here observable behavior is interface in my previous terms

Implementation should be private and only for-user-methods are visible

- User should achieve his goal with one method call, or there is a leak of implementation details
-

Encapsulation - prevent invariants violation(invariant is smth that should be true all the time)

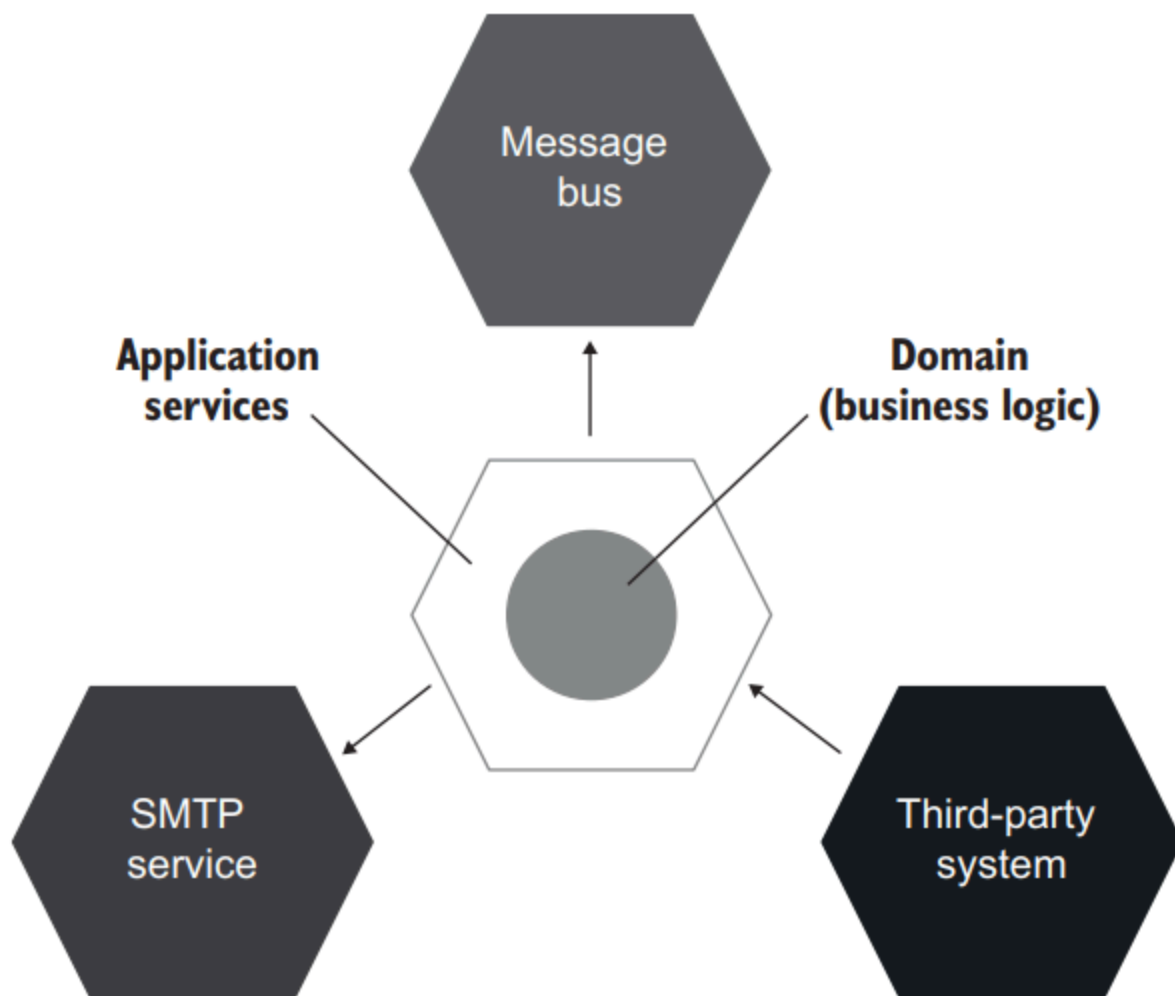
To achieve encapsulation:

- Bundle data with functions that operates with this data
- Hide implementation details from user providing only those methods that he needs

You cannot trust yourself to do the right thing all the time—so, eliminate the very possibility of doing the wrong thing

The relationship between mocks and test fragility

Hexagonal architecture



Goals of hexagonal arch:

- **To** separate concerns of application layer and domain layer, Where domain contains business logic(the-how's) and application layer manage request from client (the-what's)
 - **Domain layer** is responsible only for business logic(the-how's)
 - **Application services responsibility** to adapt the domain layer by translating the incoming requests into operations on domain classes and then persisting the results or returning them back to the caller.(+is communicating with external applications and retrieving data from the database)The application services shouldn't contain any business logic.
- **To** separate concerns you should isolate domain layer so he can't maintain different services as application layer does. Domain class only depends on other domain classes
- **To** manage External applications calls through application layer, no one has access to domain directly

Each layer of your application exhibits observable behavior and contains its own set of implementation details

Intra-system vs. inter-system communications

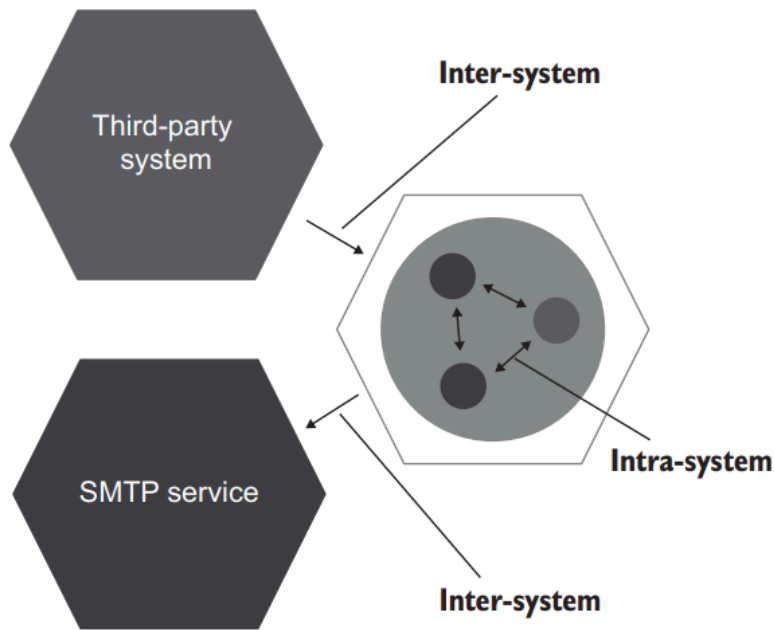


Figure 5.11 There are two types of communications: intra-system (between classes inside the application) and inter-system (between applications).

NOTE Intra-system communications are implementation details; inter-system communications are not.

Inter-system communications should be mocked and examined in tests because they are the part of interface functionality

Remember that refactoring doesn't change functionality so if external calls are included into a method, they will be always remaining after refactoring

```
[Fact]
public void Successful_purchase()
{
    var mock = new Mock<IEmailGateway>();
    var sut = new CustomerController(mock.Object);

    bool isSuccess = sut.Purchase(
        customerId: 1, productId: 2, quantity: 5);

    Assert.True(isSuccess);
    mock.Verify(
        x => x.SendReceipt(
            "customer@email.com", "Shampoo", 5),
        Times.Once);
}
```

Verifies that the system sent a receipt about the purchase

	Isolation of	A unit is	Uses test doubles for
London school	Units	A class	All but immutable dependencies
Classical school	Unit tests	A class or a set of classes	Shared dependencies

- *Shared dependency*—A dependency shared by tests (not production code)
 - *Out-of-process dependency*—A dependency hosted by a process other than the program's execution process (for example, a database, a message bus, or an SMTP service)
 - *Private dependency*—Any dependency that is not shared
- Not all shared dependencies must be mocked - if shared dependency not out of process you can initialise it before every test
 - Not all out of process dependencies should be mocked - only when they are the part of visible behavior(interface contract)
 - Using mocks to assert intra-system communications leads to *fragile* tests. Mocking is legitimate only when it's used for inter-system communications—communications that cross the application boundary—and only when the side effects of those communications are visible to the external world.

(But you can use stubs to get data you need for test, just don't verify calls)

(no you can't, you need test db only in integration tests, thus you need to use real db)

Styles of unit testing

This chapter covers

- Comparing styles of unit testing
- The relationship between functional and hexagonal architectures
- Transitioning to output-based testing

Summary

- *Output-based* testing is a style of testing where you feed an input to the SUT and check the output it produces. This style of testing assumes there are no hidden inputs or outputs, and the only result of the SUT's work is the value it returns.
- *State-based* testing verifies the state of the system after an operation is completed.
- In *communication-based* testing, you use mocks to verify communications between the system under test and its collaborators.
- The classical school of unit testing prefers the state-based style over the communication-based one. The London school has the opposite preference. Both schools use output-based testing.
- Output-based testing produces tests of the highest quality. Such tests rarely couple to implementation details and thus are resistant to refactoring. They are also small and concise and thus are more maintainable.
- State-based testing requires extra prudence to avoid brittleness: you need to make sure you don't expose a private state to enable unit testing. Because state-based tests tend to be larger than output-based tests, they are also less maintainable. Maintainability issues can sometimes be mitigated (but not eliminated) with the use of helper methods and value objects.
- Communication-based testing also requires extra prudence to avoid brittleness. You should only verify communications that cross the application boundary and whose side effects are visible to the external world. Maintainability of

communication-based tests is worse compared to output-based and state-based tests. Mocks tend to occupy a lot of space, and that makes tests less readable.

- *Functional programming* is programming with mathematical functions.
- A *mathematical function* is a function (or method) that doesn't have any hidden inputs or outputs. Side effects and exceptions are hidden outputs. A reference to an internal or external state is a hidden input. Mathematical functions are explicit, which makes them extremely testable.
- The goal of functional programming is to introduce a separation between business logic and side effects.
- Functional architecture helps achieve that separation by pushing side effects to the edges of a business operation. This approach maximizes the amount of code written in a purely functional way while minimizing code that deals with side effects.
- Functional architecture divides all code into two categories: functional core and mutable shell. The *functional core* makes decisions. The *mutable shell* supplies input data to the functional core and converts decisions the core makes into side effects.
- The difference between functional and hexagonal architectures is in their treatment of side effects. Functional architecture pushes *all* side effects out of the domain layer. Conversely, hexagonal architecture is fine with side effects made by the domain layer, as long as they are limited to that domain layer only. Functional architecture is hexagonal architecture taken to an extreme.
- The choice between a functional architecture and a more traditional one is a trade-off between performance and code maintainability. Functional architecture concedes performance for maintainability gains.
- Not all code bases are worth converting into functional architecture. Apply functional architecture strategically. Take into account the complexity and the importance of your system. In code bases that are simple or not that important, the initial investment required for functional architecture won't pay off.

Refactoring toward valuable unit tests

This chapter covers

- Recognizing the four types of code
- Understanding the Humble Object pattern
- Writing valuable tests

In chapter 1, I defined the properties of a good unit test suite:

- It is integrated into the development cycle.
- It targets only the most important parts of your code base.
- It provides maximum value with minimum maintenance costs. To achieve this last attribute, you need to be able to:
 - Recognize a valuable test (and, by extension, a test of low value).
 - Write a valuable test.

Chapter 4 covered the topic of recognizing a valuable test using the four attributes: protection against regressions, resistance to refactoring, fast feedback, and maintainability. And chapter 5 expanded on the most important one of the four: resistance to refactoring

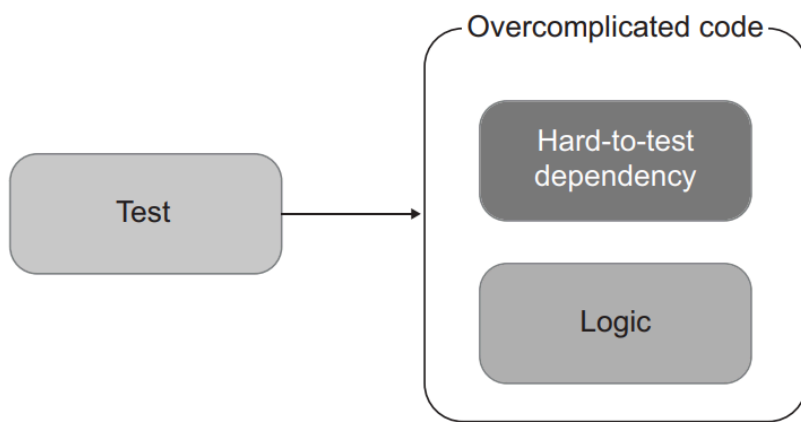


Figure 7.3 It's hard to test code that couples to a difficult dependency. Tests have to deal with that dependency, too, which increases their maintenance cost.

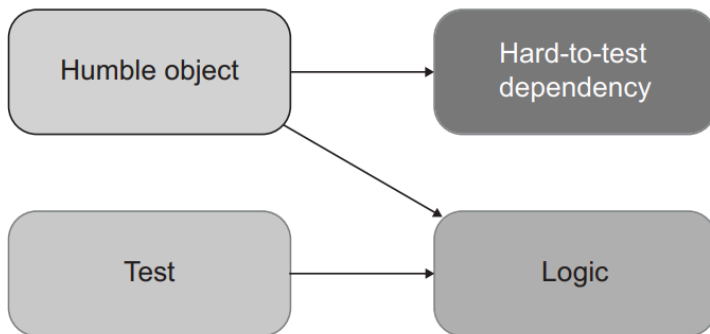


Figure 7.4 The Humble Object pattern extracts the logic out of the overcomplicated code, making that code so humble that it doesn't need to be tested. The extracted logic is moved into another class, decoupled from the hard-to-test dependency.

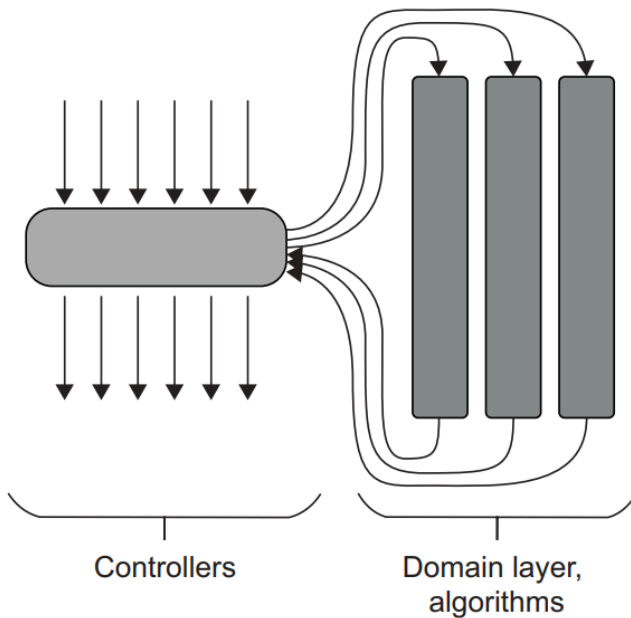
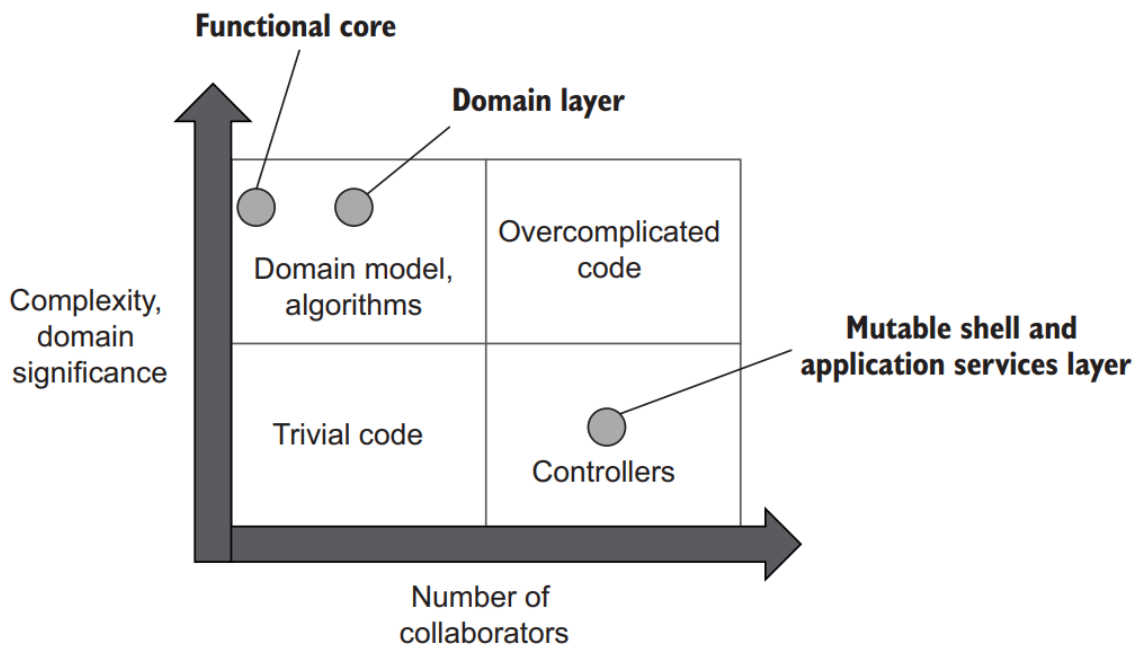


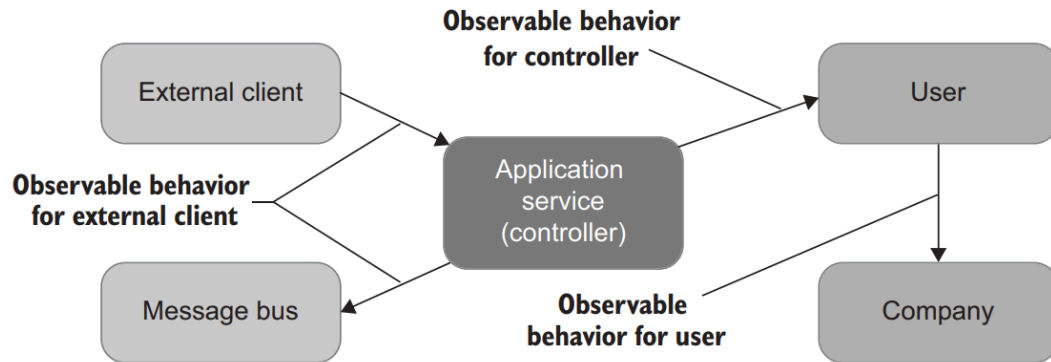
Figure 7.6 Code depth versus code width is a useful metaphor to apply when you think of the separation between the business logic and orchestration responsibilities. Controllers orchestrate many dependencies (represented as arrows in the figure) but aren't complex on their own (complexity is represented as block height). Domain classes are the opposite of that.



I can't stress enough how important this separation is. In fact, many well-known principles and patterns can be described as a form of the Humble Object pattern: they are designed specifically to segregate complex code from the code that does orchestration.

You need :

- Keep code either complex or with many dependencies
- In controller or in any layer you should have a button to do some stuff without diving into implementation details or at least a couple of buttons
- All tools should be encapsulated so client cant press button to break everything or dont press button and break everything



domain events later

- *Splitting the decision-making process into more granular steps*—Is a trade-off with the best set of pros and cons. You can mitigate the growth of controller complexity using the following two patterns:
 - The CanExecute/Execute pattern introduces a `CanDo()` for each `Do()` method and makes its successful execution a precondition for `Do()`. This pattern essentially eliminates the controller's decision-making because there's no option not to call `CanDo()` before `Do()`.

```

public string CanChangeEmail()
{
    if (IsEmailConfirmed)
        return "Can't change a confirmed email";

    return null;
}

public void ChangeEmail(string newEmail, Company company)
{
    Precondition.Requires(CanChangeEmail() == null);

    /* the rest of the method */
}

```

(can write precondition for more readability)

- Domain events help track important changes in the domain model, and then convert those changes to calls to out-of-process dependencies. This pattern removes the tracking responsibility from the controller.
- It's easier to test abstractions than the things they abstract. Domain events are abstractions on top of upcoming calls to out-of-process dependencies. Changes in domain classes are abstractions on top of upcoming modifications in the data storage.

Integration testing

Why integration testing?

This chapter covers

- Understanding the role of integration testing
- Diving deeper into the Test Pyramid concept
- Writing valuable integration tests

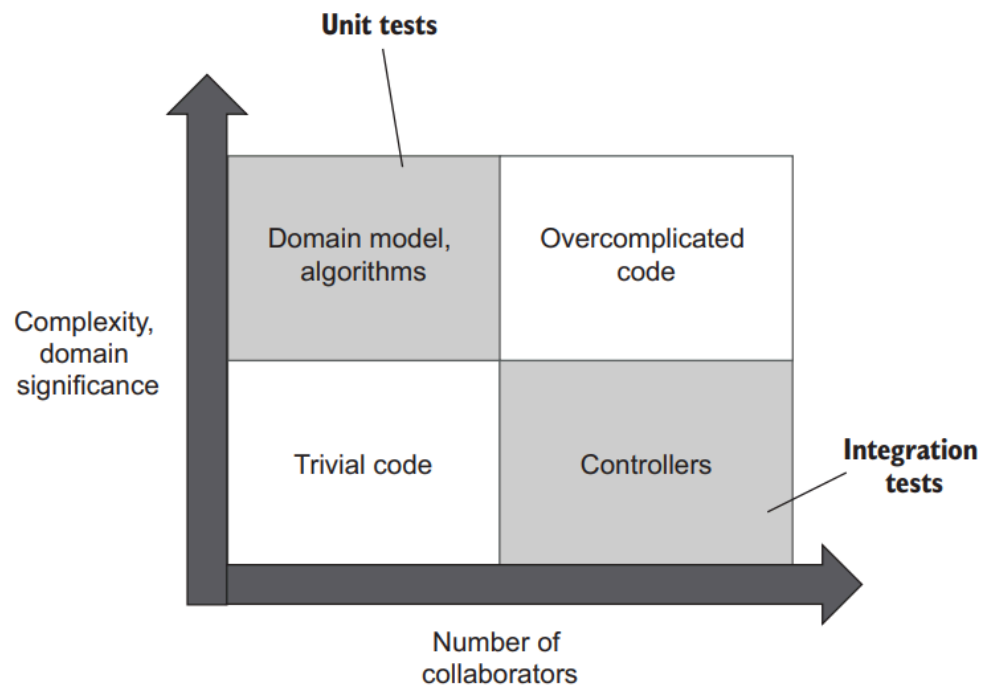


Figure 8.1 Integration tests cover controllers, while unit tests cover the domain model and algorithms. Trivial and overcomplicated code shouldn't be tested at all.

The ratio between unit and integration tests can differ depending on the project's specifics, but the general rule of thumb is the following: check as many of the business scenario's edge cases as possible with unit tests; use integration tests to cover one happy path, as well as any edge cases that can't be covered by unit tests.

DEFINITION A *happy path* is a successful execution of a business scenario. An *edge case* is when the business scenario execution results in an error.

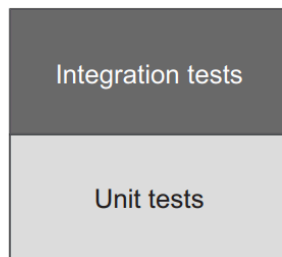
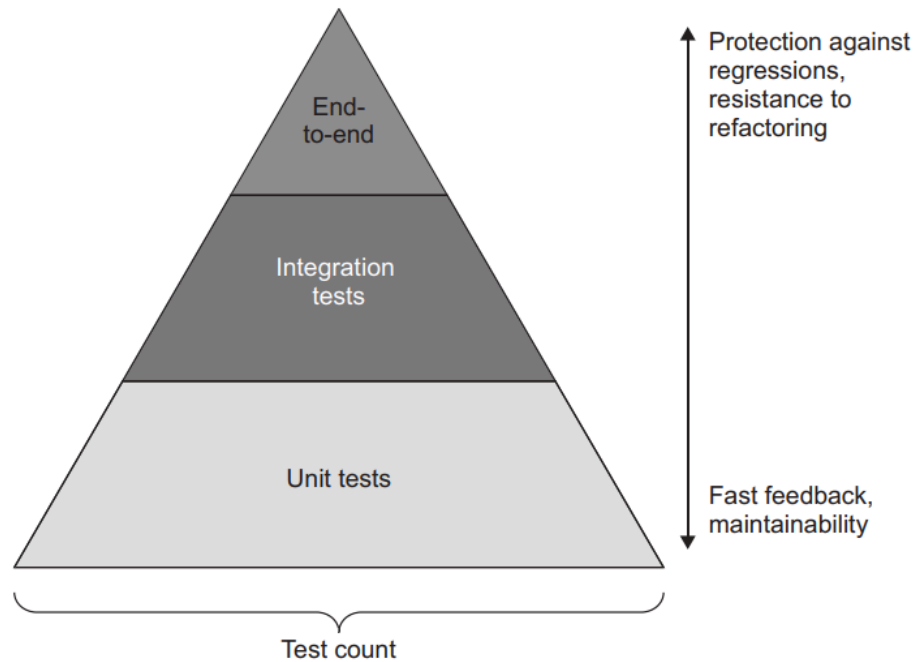


Figure 8.3 The Test Pyramid of a simple project. Little complexity requires a smaller number of unit tests compared to a normal pyramid.

You don't need to do integration tests of edge cases that immediately become noticeable, maybe on first run

TIP It's better to not write a test at all than to write a bad test. A test that doesn't provide significant value is a *bad* test.

The Fail Fast principle

The *Fail Fast principle* stands for stopping the current operation as soon as any unexpected error occurs. This principle makes your application more stable by

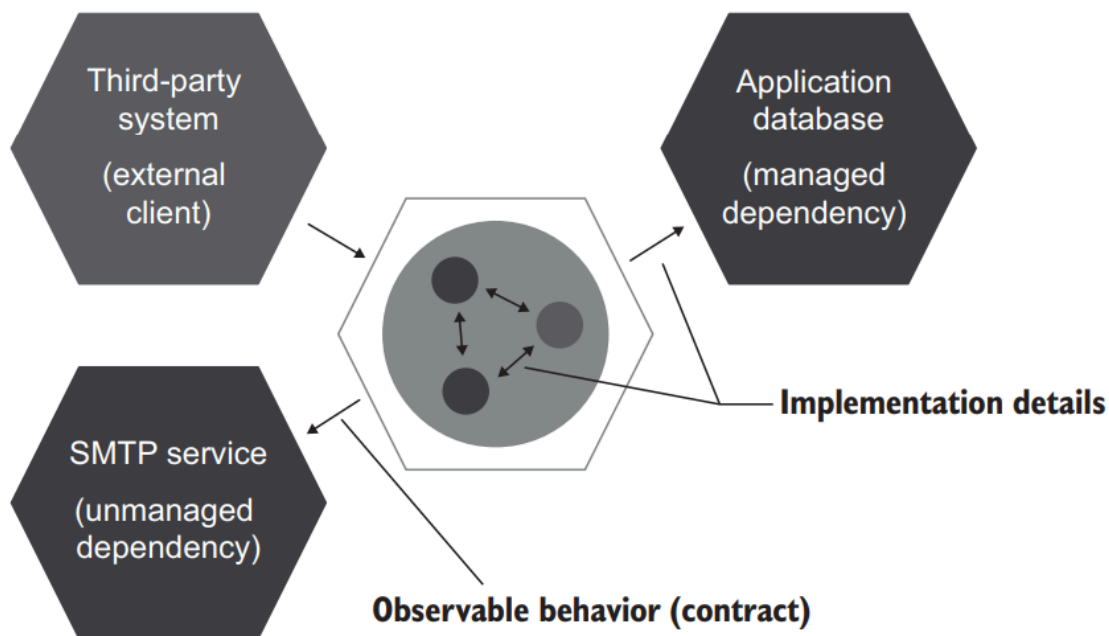
- *Shortening the feedback loop*—The sooner you detect a bug, the easier it is to fix. A bug that is already in production is orders of magnitude more expensive to fix compared to a bug found during development.
- *Protecting the persistence state*—Bugs lead to corruption of the application's state. Once that state penetrates into the database, it becomes much harder to fix. Failing fast helps you prevent the corruption from spreading.

Which out-of-process dependencies to test directly

All out-of-process dependencies fall into two categories:

- *Managed dependencies (out-of-process dependencies you have full control over)*—These dependencies are only accessible through your application; interactions with them aren't visible to the external world. A typical example is a database. External systems normally don't access your database directly; they do that through the API your application provides.
- *Unmanaged dependencies (out-of-process dependencies you don't have full control over)*—Interactions with such dependencies *are* observable externally. Examples include an SMTP server and a message bus: both produce side effects visible to other applications.

IMPORTANT Use real instances of managed dependencies; replace unmanaged dependencies with mocks.



Dont do shared database, A better way to do the integration is via an API (for synchronous communications) or a message bus (for asynchronous communications).

But if some tables from your database already accessible for another application, you need to treat those tables as unmanages dependencie and use mocks

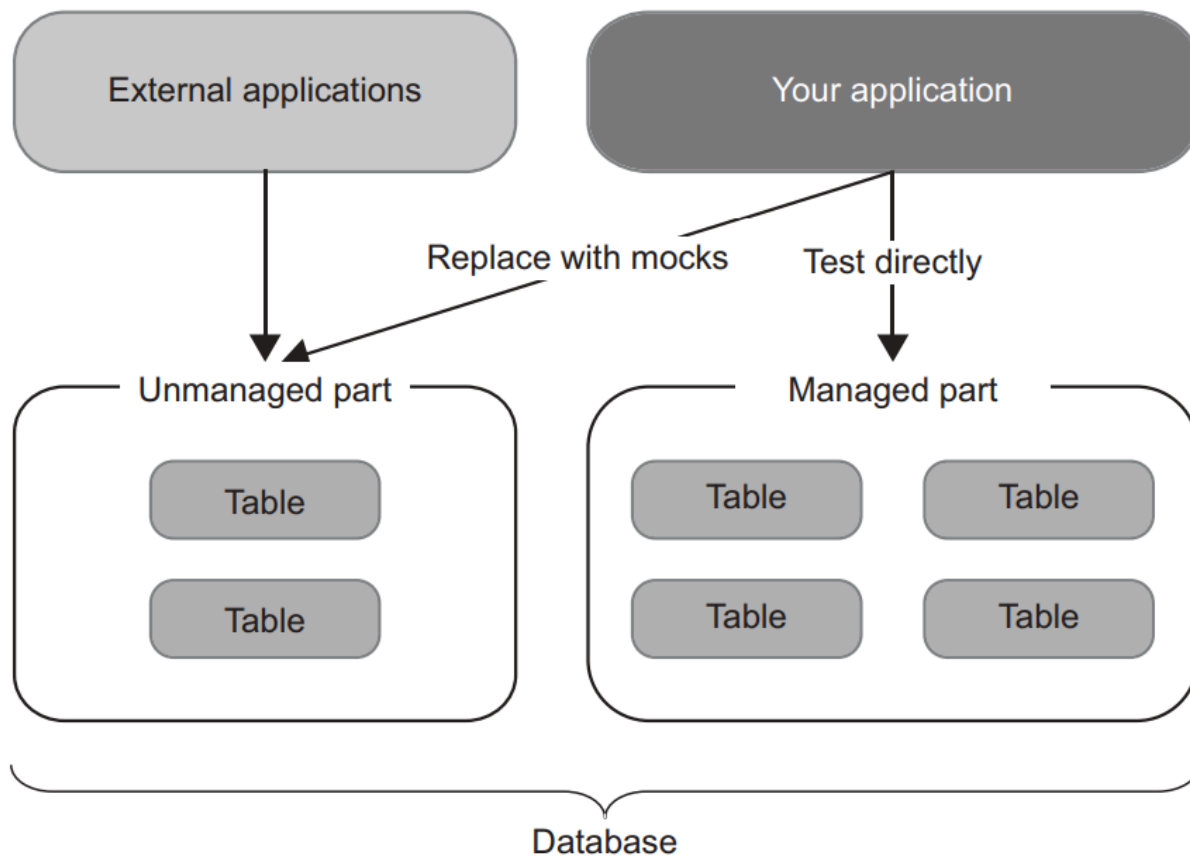


Figure 8.5 Treat the part of the database that is visible to external applications as an unmanaged dependency. Replace it with mocks in integration tests. Treat the rest of the database as a managed dependency. Verify its final state, not interactions with it.

What if you can't use a real database in integration tests?

If you can't test the database as-is, don't write integration tests at all, and instead, focus exclusively on unit testing of the domain model. Remember to always put all your tests under close scrutiny. Tests that don't provide a high enough value should have no place in your test suite.

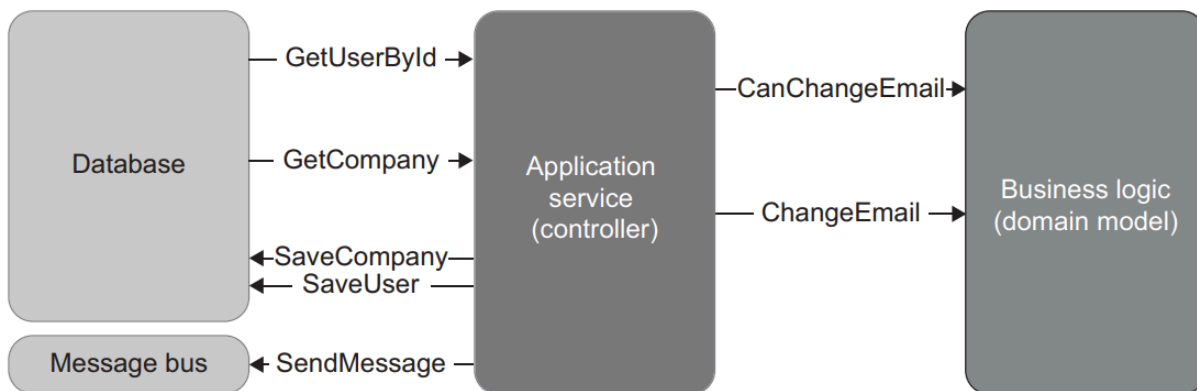


Figure 8.6 The use case of changing the user's email. The controller orchestrates the work between the database, the message bus, and the domain model.

```
public class UserController
{
    private readonly Database _database = new Database();
    private readonly MessageBus _messageBus = new MessageBus();

    public string ChangeEmail(int userId, string newEmail)
    {

        object[] userData = _database.GetUserById(userId);
        User user = UserFactory.Create(userData);

        string error = user.CanChangeEmail();
        if (error != null)
            return error;

        object[] companyData = _database.GetCompany();
        Company company = CompanyFactory.Create(companyData);

        user.ChangeEmail(newEmail, company);

        _database.SaveCompany(company);
        _database.SaveUser(user);
        foreach (EmailChangedEvent ev in user.EmailChangedEvents)
        {
            _messageBus.SendEmailChangedMessage(ev.UserId, ev.NewEmail);
        }

        return "OK";
    }
}
```

END TO END

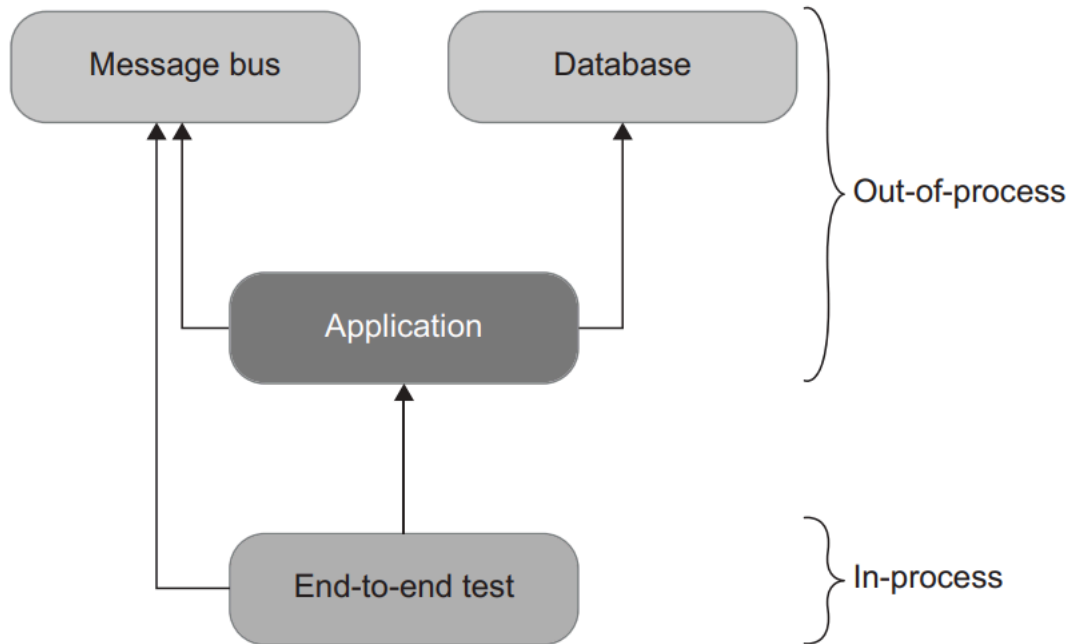


Figure 8.7 End-to-end tests emulate the external client and therefore test a deployed version of the application with all out-of-process dependencies included in the testing scope. End-to-end tests shouldn't check managed dependencies (such as the database) directly, only indirectly through the application.

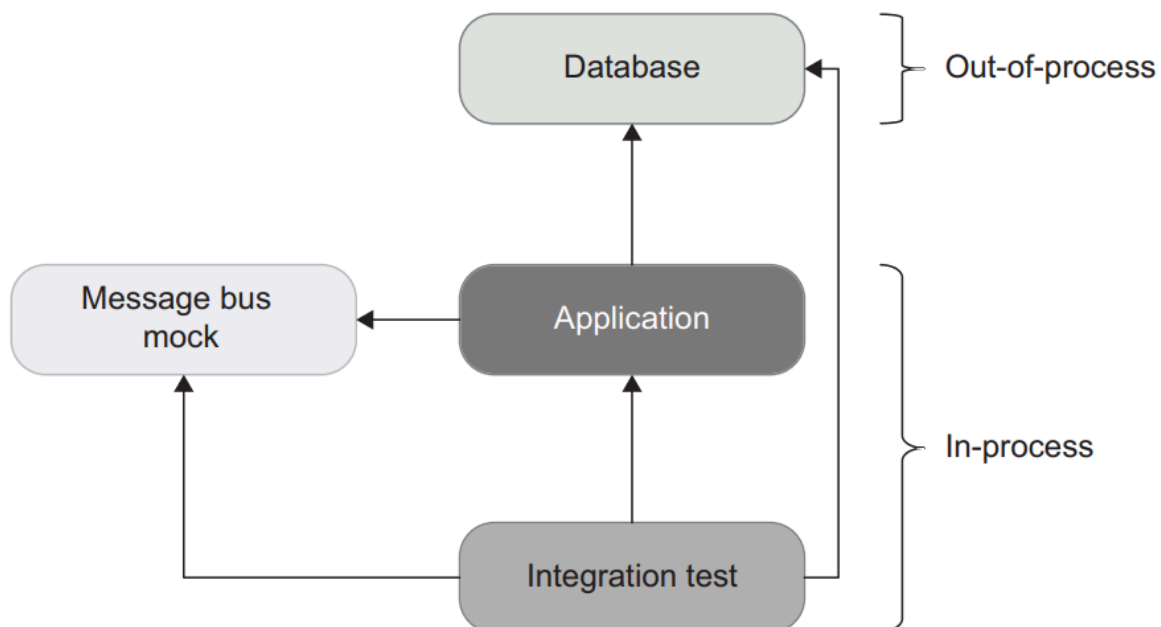


Figure 8.8 Integration tests host the application within the same process. Unlike end-to-end tests, integration tests substitute unmanaged dependencies with mocks. The only out-of-process components for integration tests are managed

Listing 8.2 The integration test

```
[Fact]
public void Changing_email_from_corporate_to_non_corporate()
{
    // Arrange
    var db = new Database(ConnectionString);
    User user = CreateUser(
        "user@mycorp.com", UserType.Employee, db);
    CreateCompany("mycorp.com", 1, db);

    var messageBusMock = new Mock<IMessageBus>();
    var sut = new UserController(db, messageBusMock.Object);

    // Act
    string result = sut.ChangeEmail(user.UserId, "new@gmail.com");

    // Assert
    Assert.Equal("OK", result);

    object[] userData = db.GetUserById(user.UserId);
    User userFromDb = UserFactory.Create(userData);
    Assert.Equal("new@gmail.com", userFromDb.Email);
    Assert.Equal(UserType.Customer, userFromDb.Type);

    object[] companyData = db.GetCompany();
    Company companyFromDb = CompanyFactory
        .Create(companyData);
    Assert.Equal(0, companyFromDb.NumberOfEmployees);

    messageBusMock.Verify(
        x => x.SendEmailChangedMessage(
            user.UserId, "new@gmail.com"),
        Times.Once);
}
```

Database repository

Creates the user and company in the database

Sets up a mock for the message bus

Asserts the user's state

Asserts the company's state

Checks the interactions with the mock

TIP Notice that in the arrange section, the test doesn't insert the user and the company into the database on its own but instead calls the `CreateUser` and `CreateCompany` helper methods. These methods can be reused across multiple integration tests.

Integration testing best practices

There are some general guidelines that can help you get the most out of your integration tests:

- Making domain model boundaries explicit
- Reducing the number of layers in the application
- Eliminating circular dependencies

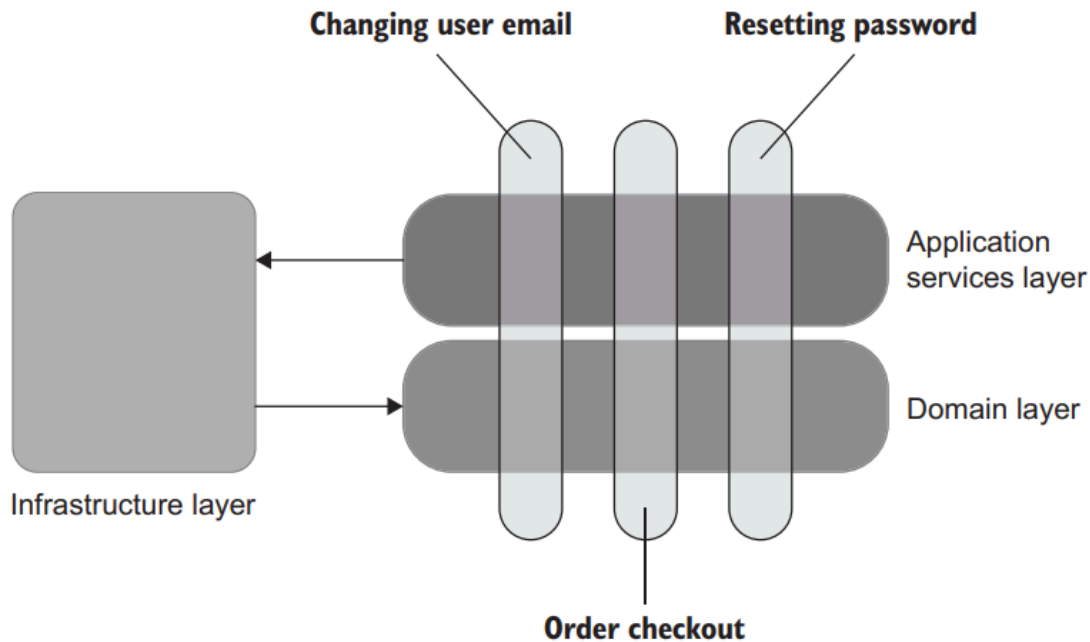


Figure 8.10 You can get away with just three layers: the domain layer (contains domain logic), application services layers (provides an entry point for the external client, and coordinates the work between domain classes and out-of-process dependencies), and infrastructure layer (works with out-of-process dependencies; database repositories, ORM mappings, and SMTP gateways reside in this layer).

Eliminating circular dependencies

Another practice that can drastically improve the maintainability of your code base and make testing easier is eliminating circular dependencies.

DEFINITION A *circular dependency* (also known as *cyclic dependency*) is two or more classes that directly or indirectly depend on each other to function properly.

Using multiple act sections in a test

Hard-to-manage out-of-process dependencies are the only legitimate reason to write a test with more than one act section. This is why you should never have multiple acts in a unit test—unit tests don't work with out-of-process dependencies. Even integration tests should rarely have several acts. In practice, multistep tests almost always belong to the category of end-to-end tests.

Use logs with parameters because this logs are "structured", can be converted in many types and more easy to read

Mocking best practices

This chapter covers

- Maximizing the value of mocks
- Replacing mocks with spies
- Mocking best practices

```
public string ChangeEmail(int userId, string newEmail)
{
    object[] userData = _database.GetUserById(userId);
    User user = UserFactory.Create(userData);

    string error = user.CanChangeEmail();
    if (error != null)
        return error;

    object[] companyData = _database.GetCompany();
    Company company = CompanyFactory.Create(companyData);

    user.ChangeEmail(newEmail, company);

    _database.SaveCompany(company);
    _database.SaveUser(user);
    _eventDispatcher.Dispatch(user.DomainEvents);

    return "OK";
}
```

Listing 9.2 Event dispatcher

```
public class EventDispatcher
{
    private readonly IMessageBus _messageBus;
    private readonly IDomainLogger _domainLogger;

    public EventDispatcher(
        IMessageBus messageBus,
        IDomainLogger domainLogger)
    {
        _domainLogger = domainLogger;
        _messageBus = messageBus;
    }

    public void Dispatch(List<IDomainEvent> events)
    {
        foreach (IDomainEvent ev in events)
        {
            Dispatch(ev);
        }
    }

    private void Dispatch(IDomainEvent ev)
    {
        switch (ev)
        {
            case EmailChangedEvent emailChangedEvent:
                _messageBus.SendEmailChangedMessage(
                    emailChangedEvent.UserId,
                    emailChangedEvent.NewEmail);
                break;

            case UserTypeChangedEvent userTypeChangedEvent:
                _domainLogger.UserTypeHasChanged(
                    userTypeChangedEvent.UserId,
                    userTypeChangedEvent.OldType,
                    userTypeChangedEvent.NewType);
                break;
        }
    }
}
```

TIP When mocking, always adhere to the following guideline: verify interactions with unmanaged dependencies at the very edges of your system.

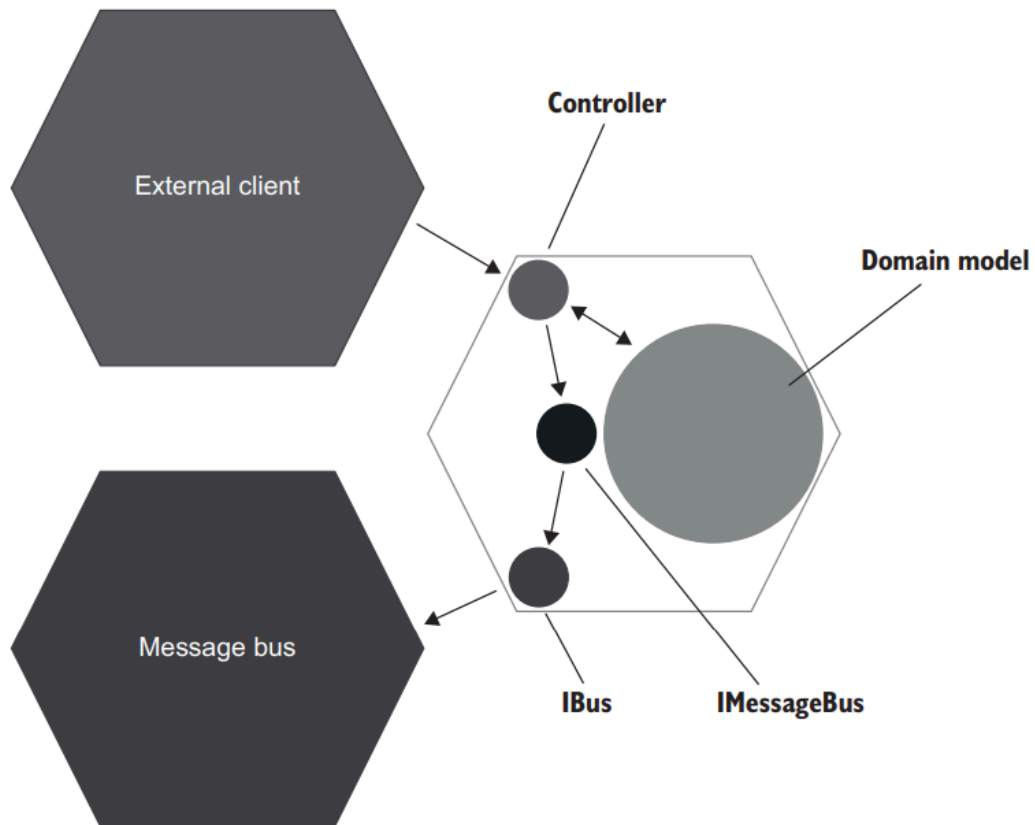


Figure 9.1 `IBus` resides at the system's edge; `IMessageBus` is only an intermediate link in the chain of types between the controller and the message bus. Mocking `IBus` instead of `IMessageBus` achieves the best protection against regressions.

- Don't rely on production code when making assertions. Use a separate set of literals and constants in tests. Duplicate those literals and constants from the production code if necessary. Tests should provide a checkpoint independent of the production code. Otherwise, you risk producing *tautology tests* (tests that don't verify anything and contain semantically meaningless assertions).
- Not all unmanaged dependencies require the same level of backward compatibility. If the exact structure of the message isn't important, and you only want to verify the existence of that message and the information it carries, you can ignore the guideline of verifying interactions with unmanaged dependencies at the very edges of your system. The typical example is logging.

Mocking best practices

You've learned two major mocking best practices so far:

- Applying mocks to unmanaged dependencies only
- Verifying the interactions with those dependencies at the very edges of your system

In this section, I explain the remaining best practices:

- Using mocks in integration tests only, not in unit tests
- Always verifying the number of calls made to the mock
- Mocking only types that you own

In unit tests you shouldn't use mocks because unit tests cover business logic with no interactions with out-of-process, inter-system dependencies, but you should mock only unmanaged dependencies, thus you don't need to use mocks in unit tests

Mocking only types that you own

- You often don't have a deep understanding of how the third-party code works.
- Even if that code already provides built-in interfaces, it's risky to mock those interfaces, because you have to be sure the behavior you mock matches what the external library actually does.
- Adapters abstract non-essential technical details of the third-party code and define the relationship with the library in your application's terms.

I fully agree with this analysis. Adapters, in effect, act as an anti-corruption layer between your code and the external world.² These help you to

- Abstract the underlying library's complexity
- Only expose features you need from the library
- Do that using your project's domain language

Note that the “mock your own types” guideline *doesn't* apply to in-process dependencies. As I explained previously, mocks are for unmanaged dependencies only. Thus, there's no need to abstract in-memory or managed dependencies. For instance, if a library provides a date and time API, you can use that API as-is, because it doesn't reach out to unmanaged dependencies. Similarly, there's no need to abstract an ORM as long as it's used for accessing a database that isn't visible to external applications. Of course, you can introduce your own wrapper on top of any library, but it's rarely worth the effort for anything other than unmanaged dependencies.

Testing the database

This chapter covers

- Prerequisites for testing the database
- Database testing best practices
- Test data life cycle
- Managing database transactions in tests

Prerequisites for testing the database

- Keeping the database in the source control system
- Using a separate database instance for every developer
- Applying the migration-based approach to database delivery

Keeping the database in the source control system(git)

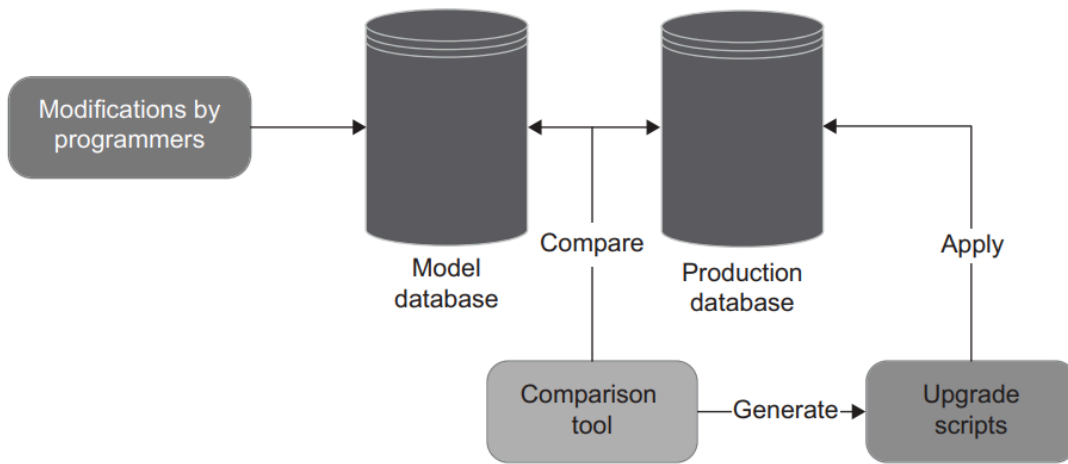


Figure 10.1 Having a dedicated instance as a model database is an anti-pattern. The database schema is best stored in a source control system.

Using a model database is a horrible way to maintain database schema. That’s because there’s

- *No change history*—You can’t trace the database schema back to some point in the past, which might be important when reproducing bugs in production.
- *No single source of truth*—The model database becomes a competing source of truth about the state of development. Maintaining two such sources (Git and the model database) creates an additional burden.

Reference data(userType,adress,country,all stuff that you cant modify with your application,and on wich your regular data has foreign keys)is part of your database schema

Because reference data is essential for your application, you should keep it in the source control system along with tables, views, and other parts of the database schema, in the form of SQL INSERT statements.

Dont use comparison tools for changes of db.Use migrations.

TIP Apply every modification to the database schema (including reference data) through migrations. Don’t modify migrations once they are committed to the source control. If a migration is incorrect, create a new migration instead of fixing the old one. Make exceptions to this rule only when the incorrect migration can lead to data loss.

Database transaction management

Managing database transactions in production code

Listing 10.4 User controller with Entity Framework

```
public class UserController
{
    private readonly CrmContext _context;
    private readonly UserRepository _userRepository;
    private readonly CompanyRepository _companyRepository;
    private readonly EventDispatcher _eventDispatcher;

    public UserController(
        CrmContext context,
        MessageBus messageBus,
        IDomainLogger domainLogger)
    {
        _context = context;
        _userRepository = new UserRepository(
            context);
        _companyRepository = new CompanyRepository(
            context);
        _eventDispatcher = new EventDispatcher(
            messageBus, domainLogger);
    }

    public string ChangeEmail(int userId, string newEmail)
    {
        User user = _userRepository.GetUserById(userId);

        string error = user.CanChangeEmail();
        if (error != null)
            return error;

        Company company = _companyRepository.GetCompany();

        user.ChangeEmail(newEmail, company);

        _companyRepository.SaveCompany(company);
        _userRepository.SaveUser(user);
        _eventDispatcher.Dispatch(user.DomainEvents);

        _context.SaveChanges();
        return "OK";
    }
}
```

extension.

CrmContext replaces Transaction.

Data inconsistencies in non-relational databases

It's easy to avoid data inconsistencies when using a relational database: all major relational databases provide atomic updates that can span as many rows as needed. But how do you achieve the same level of protection with a non-relational database such as MongoDB?

The problem with most non-relational databases is the lack of transactions in the classical sense; atomic updates are guaranteed only within a single document. If a business operation affects multiple documents, it becomes prone to inconsistencies. (In non-relational databases, a *document* is the equivalent of a *row*.)

Non-relational databases approach inconsistencies from a different angle: they require you to design your documents such that no business operation modifies more than one of those documents at a time. This is possible because documents are more flexible than rows in relational databases. A single document can store data of any shape and complexity and thus capture side effects of even the most sophisticated business operations.

(continued)

In domain-driven design, there's a guideline saying that you shouldn't modify more than one aggregate per business operation. This guideline serves the same goal: protecting you from data inconsistencies. The guideline is only applicable to systems that work with document databases, though, where each document corresponds to one aggregate.

Managing database transactions in integration tests

Listing 10.14 Integration test after moving all technicalities out of it

```
public class UserControllerTests : IntegrationTests
{
    [Fact]
    public void Changing_email_from_corporate_to_non_corporate()
    {
        // Arrange
        ➤ User user = CreateUser(
            email: "user@mycorp.com",
            type: UserType.Employee);
        ➤ CreateCompany("mycorp.com", 1);

        var busSpy = new BusSpy();
        var messageBus = new MessageBus(busSpy);
        var loggerMock = new Mock<IDomainLogger>();

        // Act
        ➤ string result = Execute(
            x => x.ChangeEmail(user.UserId, "new@gmail.com"),
            messageBus, loggerMock.Object);

        // Assert
        Assert.Equal("OK", result);

        ➤ User userFromDb = QueryUser(user.UserId);
        userFromDb
            .ShouldExist()
            .WithEmail("new@gmail.com")
            .WithType(UserType.Customer);

        ➤ Company companyFromDb = QueryCompany();
        companyFromDb
            .ShouldExist()
            .WithNumberOfEmployees(0);
    }
}
```

```
busSpy.ShouldSendNumberOfMessages(1)
    .WithEmailChangedMessage(user.UserId, "new@gmail.com");
loggerMock.Verify(
```

CHAPTER 10 *Testing the database*

```
x => x.UserTypeHasChanged(
    user.UserId, UserType.Employee, UserType.Customer),
Times.Once);
```

TIP Use at least three transactions or units of work in an integration test: one per each arrange, act, and assert section.

Test data life cycle

The shared database raises the problem of isolating integration tests from each other. To solve this problem, you need to

- Execute integration tests sequentially.
- Remove leftover data between test runs.

Overall, your tests shouldn't depend on the state of the database. Your tests should bring that state to the required condition on their own.

Listing 10.6 Base class for integration tests

```
public abstract class IntegrationTests
{
    private const string ConnectionString = "...";

    protected IntegrationTests()
    {
        ClearDatabase();
    }

    private void ClearDatabase()
    {
        string query =
            "DELETE FROM dbo.[User];" +
            "DELETE FROM dbo.Company;";

        using (var connection = new SqlConnection(ConnectionString))
        {
            var command = new SqlCommand(query, connection)
            {
                CommandType = CommandType.Text
            };

            connection.Open();
            command.ExecuteNonQuery();
        }
    }
}
```

Deletion script

TIP The deletion script must remove all regular data but none of the reference data. Reference data, along with the rest of the database schema, should be controlled solely by migrations.

Listing 11.4 A class with private state

```
public class Customer
{
    private CustomerStatus _status =
        CustomerStatus.Regular;

    public void Promote()
    {
        _status = CustomerStatus.Preferred;
    }

    public decimal GetDiscount()
    {
        return _status == CustomerStatus.Preferred ? 0.05m : 0m;
    }
}
```

Private state

That would be an anti-pattern, however. Remember, *your tests should interact with the system under test (SUT) exactly the same way as the production code and shouldn't have any special privileges*. In listing 11.4, the `_status` field is hidden from the production code and thus is not part of the SUT's observable behavior. Exposing that field would result in coupling tests to implementation details. How to test `Promote()`, then?

What you should do, instead, is look at how the production code uses this class. In this particular example, the production code doesn't care about the customer's status; otherwise, that field would be public. The only information the production code does care about is the discount the customer gets after the promotion. And so that's what you need to verify in tests. You need to check that

- A newly created customer has no discount.
- Once the customer is promoted, the discount becomes 5%.

NOTE Widening the public API surface for the sake of testability is a bad practice.

Leaking domain knowledge to tests

```
public class CalculatorTests
{
    [Fact]
    public void Adding_two_numbers()
    {
        int value1 = 1;
        int value2 = 3;
        int expected = value1 + value2;    <— The leakage

        int actual = Calculator.Add(value1, value2);

        Assert.Equal(expected, actual);
    }
}

public class CalculatorTests
{
    [Theory]
    [InlineData(1, 3, 4)]
    [InlineData(11, 33, 44)]
    [InlineData(100, 500, 600)]
    public void Adding_two_numbers(int value1, int value2, int expected)
    {
        int actual = Calculator.Add(value1, value2);
        Assert.Equal(expected, actual);
    }
}
```

It can seem counterintuitive at first, but hardcoding the expected result is a good practice when it comes to unit testing. The important part with the hardcoded values is to precalculate them using something other than the SUT, ideally with the help of a domain expert. Of course, that's only if the algorithm is complex enough (we are all

Code pollution

The next anti-pattern is code pollution.

DEFINITION *Code pollution* is adding production code that's only needed for testing.

```
public class Logger
{
    private readonly bool _isTestEnvironment;

    public Logger(bool isTestEnvironment)    ← The switch
    {
        _isTestEnvironment = isTestEnvironment;
    }

    public interface ILogger
    {
        void Log(string text);
    }
}
```

```
public class Logger : ILogger
{
    public void Log(string text)
    {
        /* Log the text */
    }
}
```

**Belongs in the
production code**

```
public class FakeLogger : ILogger
{
    public void Log(string text)
    {
        /* Do nothing */
    }
}
```

**Belongs in
the test code**

Mocking concrete classes

Listing 11.11 A class that calculates statistics

```
public class StatisticsCalculator
{
    public (double totalWeight, double totalCost) Calculate(
        int customerId)
    {
        List<DeliveryRecord> records = GetDeliveries(customerId);

        double totalWeight = records.Sum(x => x.Weight);
        double totalCost = records.Sum(x => x.Cost);

        return (totalWeight, totalCost);
    }

    public List<DeliveryRecord> GetDeliveries(int customerId)
    {
        /* Call an out-of-process dependency
        to get the list of deliveries */
    }
}
```

Listing 11.12 A controller using StatisticsCalculator

```
public class CustomerController
{
    private readonly StatisticsCalculator _calculator;

    public CustomerController(StatisticsCalculator calculator)
    {
        _calculator = calculator;
    }

    public string GetStatistics(int customerId)
    {
        (double totalWeight, double totalCost) = _calculator
            .Calculate(customerId);

        return
            $"Total weight delivered: {totalWeight}. " +
            $"Total cost: {totalCost}";
    }
}
```

Listing 11.13 Test that mocks the concrete class

```
[Fact]
public void Customer_with_no_deliveries()
{
    // Arrange
    var stub = new Mock<StatisticsCalculator> { CallBase = true };
    stub.Setup(x => x.GetDeliveries(1))
        .Returns(new List<DeliveryRecord>());
    var sut = new CustomerController(stub.Object);

    // Act
    string result = sut.GetStatistics(1);

    // Assert
    Assert.Equal("Total weight delivered: 0. Total cost: 0", result);
}
```

← **GetDeliveries() must be made virtual.**

NOTE The necessity to mock a concrete class in order to preserve part of its functionality is a result of violating the Single Responsibility principle.

StatisticsCalculator combines two unrelated responsibilities: communicating with the unmanaged dependency and calculating statistics. Look at listing 11.11 again. The Calculate() method is where the domain logic lies. GetDeliveries() just gathers the inputs for that logic. Instead of mocking StatisticsCalculator, split this class in two, as the following listing shows.

```
public class DeliveryGateway : IDeliveryGateway
{
    public List<DeliveryRecord> GetDeliveries(int customerId)
    {
        /* Call an out-of-process dependency
        to get the list of deliveries */
    }
}

public class StatisticsCalculator
{
    public (double totalWeight, double totalCost) Calculate(
        List<DeliveryRecord> records)
    {
        double totalWeight = records.Sum(x => x.Weight);
        double totalCost = records.Sum(x => x.Cost);

        return (totalWeight, totalCost);
    }
}
```

Working with time

Time in assertions can be different than in the act, so there are 3 options:

1. Ambient context(Anti-pattern)
Ne ponel tam nu ladno corоче